Practica I
(Circuitos combinacionales y diseño)

Alex Rafael Soriano Severino
Mecatronica
ITLA
La Caleta, Santo Domingo
20240684@itla.edu.do

En resumen— En esta práctica se desarrollaron distintos circuitos combinacionales utilizando compuertas lógicas básicas, mapas de Karnaugh y descripciones en HDL (Verilog). Se implementaron y simularon diez ejercicios que abarcan desde funciones booleanas básicas hasta sistemas más complejos, como detectores de paridad, generadores de sucesión Fibonacci, multiplicadores binarios y controladores de semáforos. El propósito principal fue fortalecer la comprensión del diseño lógico mediante el uso de herramientas digitales como Multisim para la simulación de los diagramas lógicos y GoWin IDE para la descripción estructural y verificación en FPGA. Finalmente, se comprobó experimentalmente que los resultados teóricos y las simulaciones coinciden con el comportamiento real de los circuitos.

Abstract— In this practice, several combinational circuits were designed and simulated using basic logic gates, Karnaugh maps, and HDL (Verilog) descriptions. Ten exercises were implemented, ranging from basic Boolean functions to more complex systems such as parity detectors, Fibonacci sequence generators, binary multipliers, and traffic light controllers. The main objective was to strengthen the understanding of logical design through the use of digital tools such as Multisim for logic diagram simulation and GoWin IDE for structural implementation and FPGA verification. Experimental results confirmed the accuracy of the theoretical analysis and demonstrated the expected behavior of all combinational systems.

Palabras claves— Electrónica digital, compuertas lógicas, circuito combinacional, tabla de verdad, mapa de Karnaugh, Verilog, FPGA, HDL, simulación digital.

I. INTRODUCCION

En esta práctica se abordó el diseño y la implementación de diferentes sistemas combinacionales utilizando compuertas lógicas básicas, álgebra booleana y herramientas de simulación y programación digital. A través de los distintos ejercicios se buscó reforzar conceptos fundamentales como la tabla de verdad, la simplificación mediante mapas de Karnaugh, las formas canónicas **Suma de Productos (SoP)** y **Producto de Sumas (PoS)**, y la aplicación práctica de estos en **Verilog** sobre **FPGA**.

El desarrollo de la práctica permitió aplicar los conocimientos teóricos en entornos reales de simulación (Multisim) e implementación (GoWin), comprendiendo así el proceso completo de diseño: desde la formulación de la función lógica hasta la validación de su comportamiento físico. Los circuitos realizados abarcaron temas esenciales del diseño digital, como paridad, sucesión Fibonacci, multiplicación binaria, complemento a dos y sistemas de control lógicos, consolidando así una visión integral del diseño combinacional.

II. MARCO TEÓRICO

A. Compuertas Lógicas

Las compuertas lógicas son bloques fundamentales de la electrónica digital. Cada compuerta realiza una función booleana básica:

- **AND (Y):** su salida es 1 únicamente cuando todas sus entradas son 1.
- **OR (O):** su salida es 1 cuando al menos una entrada es 1.
- **NOT (NO):** invierte el estado de la entrada (0 a 1, 1 a 0).

Estas compuertas se implementan en circuitos integrados como la familia 74HCxx o equivalentes, y permiten construir sistemas lógicos combinacionales y secuenciales.

B. Tabla de Verdad

La tabla de verdad es una herramienta que muestra todas las posibles combinaciones de entradas y las salidas resultantes. Es esencial para diseñar, verificar y comprobar el comportamiento de un circuito lógico.

C. Circuitos Combinacionales

Un circuito combinacional es aquel en el que las salidas dependen únicamente de las entradas actuales, sin necesidad de memoria o almacenamiento. Estos circuitos son empleados en sistemas de decisión, control y procesamiento digital.

D. Formas Canónicas (SoP y PoS)

Las formas canónicas son representaciones estándar de las funciones booleanas:

- **Suma de Productos (SoP):** combina minterminos mediante sumas lógicas (OR). Cada mintermino representa una combinación de entradas que produce salida 1.
- **Producto de Sumas (PoS):** combina maxterminos mediante productos lógicos (AND). Cada maxtermino representa una combinación de entradas que produce salida 0.

Estas formas permiten expresar y simplificar funciones lógicas de manera estructurada antes de implementarlas en hardware.

E. Paridad

La paridad es una propiedad que indica si el número de bits con valor 1 en una palabra binaria es par o impar.

- En paridad par, el bit de paridad se ajusta para que el total de 1’s sea par.

- En paridad impar, se ajusta para que el total sea impar.

Los detectores de paridad se usan ampliamente en sistemas de comunicación y memoria digital para verificar la integridad de los datos.

F. Sucesión de Fibonacci

La sucesión de Fibonacci es una serie numérica donde cada término es la suma de los dos anteriores (1, 1, 2, 3, 5, 8...). En electrónica digital puede implementarse usando circuitos combinacionales que representen los valores de salida correspondientes a una secuencia binaria. Este tipo de diseño permite estudiar cómo generar funciones aritméticas con compuertas lógicas.

G. Complemento a Dos

El complemento a dos es una técnica usada para representar números negativos en binario.
Se obtiene invirtiendo todos los bits del número (complemento a uno) y sumando 1 al resultado.
Su ventaja es que permite realizar sumas y restas binarias sin circuitos adicionales, ya que el hardware interpreta automáticamente el bit más significativo (MSB) como signo:

- Si MSB = 0 → número positivo
- Si MSB = 1 → número negativo

H. Multiplicadores Binarios

Los multiplicadores binarios combinan varias operaciones AND y sumadores para generar un producto a partir de dos números binarios. En un multiplicador de 2 bits por 2 bits, por ejemplo, se generan productos parciales que luego se suman para obtener el resultado final. Este tipo de circuito es una aplicación directa de los principios combinacionales.

I. Lenguaje HDL y FPGA

Los Lenguajes de Descripción de Hardware (HDL), como Verilog o VHDL, permiten describir el comportamiento y la estructura de circuitos digitales mediante código.
Las FPGAs (Field Programmable Gate Arrays) son dispositivos programables que pueden implementar físicamente los circuitos descritos en HDL, permitiendo la verificación práctica de sistemas digitales sin necesidad de construirlos con compuertas discretas.

J. *Verilog*

Verilog es un lenguaje HDL ampliamente utilizado en la industria.
Permite describir circuitos en tres niveles:

- Conductual, que define el comportamiento lógico.
- Estructural, que conecta compuertas y módulos entre sí.
- RTL (Register Transfer Level), que describe operaciones entre registros.

Su sintaxis es similar al lenguaje C y se utiliza dentro de entornos como GoWin IDE, Quartus o Vivado para crear, simular e implementar diseños digitales sobre FPGA.

III. Montaje Experimental

El trabajo experimental se desarrolló en Multisim para la simulación lógica y en GoWin IDE para la implementación en FPGA, siguiendo un proceso de diseño progresivo en diez ejercicios:

1) Diseño de circuito combinacional de 4 bits: Se elaboró una tabla de verdad y se determinó la salida activa cuando todas las entradas estaban en High, todas en Low o si había un número impar de bits activos. Se simplificó en SoP y PoS y se verificó en Multisim.

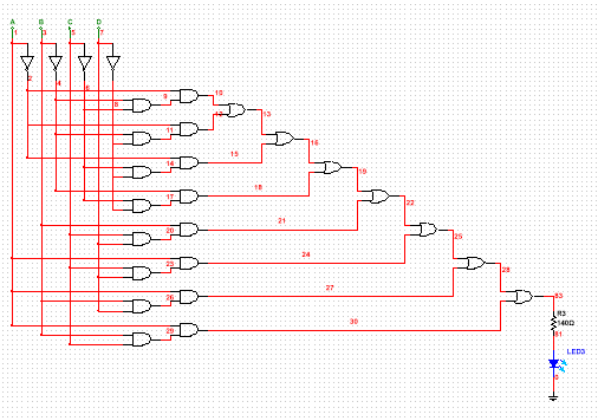
Tabla de Verdad (1)				
X3	X2	X1	X0	S
0	0	0	0	1
0	0	0	1	1
0	0	1	0	1
0	0	1	1	0
0	1	0	0	1
0	1	0	1	0
0	1	1	0	0
0	1	1	1	1
1	0	0	0	1
1	0	0	1	0
1	0	1	0	0
1	0	1	1	1
1	1	0	0	0
1	1	0	1	1
1	1	1	0	1
1	1	1	1	1

Mapa de Karmaugh				
AB/CD	00	01	11	10
00	1	1	0	1
01	1	0	1	0
11	0	1	1	1
10	1	0	1	0

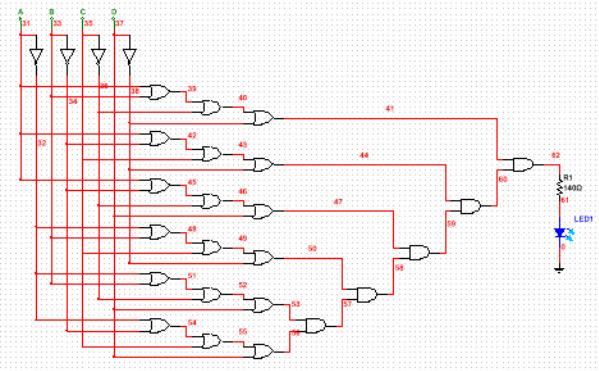
Min-terminos (SoP)
A'B'C' + A'B'D' + A'C'D' + B'C'D' + BCD + ACD + ABD + ABC

Max-terminos (PoS)
(A + B + C' + D')(A + B' + C + D')(A + B' + C' + D)(A' + B + C + D')(A' + B + C' + D)(A' + B' + C + D)

Min-terminos:



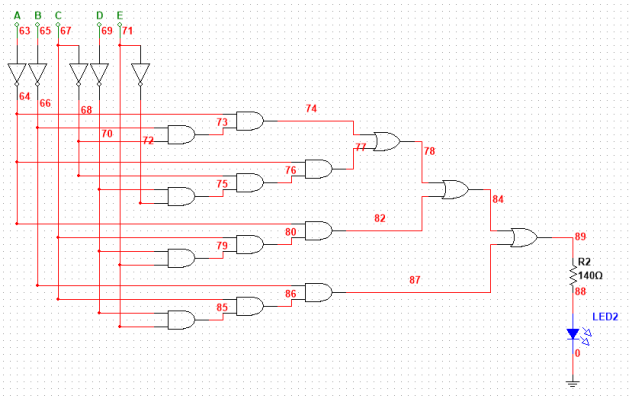
Max-terminos:



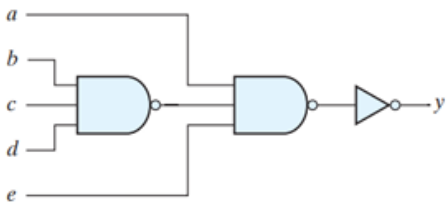
2) Generador de sucesión de Fibonacci de 5 bits: Se diseñó la tabla de verdad correspondiente a la secuencia y se obtuvo la función simplificada en SoP. El circuito se simuló con compuertas lógicas.

Tabla de Verdad (2)					
X4	X3	X2	X1	X0	S4
0	0	0	0	0	1
0	0	0	0	1	1
0	0	0	1	0	1
0	0	0	1	1	1
0	0	1	0	0	0
0	0	1	0	1	1
0	0	1	1	0	0
0	0	1	1	1	0
0	1	0	0	0	1
0	1	0	0	1	0
0	1	0	1	0	0
0	1	0	1	1	0
0	1	1	0	0	0
0	1	1	0	1	1
0	1	1	1	0	0
0	1	1	1	1	0
1	0	0	0	0	0
1	0	0	0	1	0
1	0	0	1	0	0
1	0	0	1	1	0
1	0	1	0	0	0
1	0	1	0	1	1
1	0	1	1	0	0
1	0	1	1	1	0
1	1	0	0	0	0
1	1	0	0	1	0
1	1	0	1	0	0
1	1	0	1	1	0
1	1	1	0	0	0
1	1	1	0	1	0
1	1	1	1	0	0
1	1	1	1	1	0

Min-terminos (SoP)
A'B'C'+A'C'D'E'+A'CD'E+B'CD'E



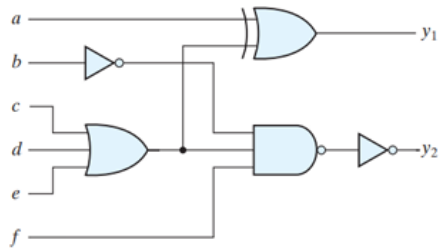
3) Deducción de funciones booleanas desde diagramas lógicos:
Se analizaron diagramas ya construidos, se derivaron sus tablas de verdad y se expresaron las funciones equivalentes.



(a)

Tabla de Verdad (3a)					
a	b	c	d	e	y
0	0	0	0	0	1
0	0	0	0	1	1
0	0	0	1	0	1
0	0	0	1	1	1
0	0	1	0	0	1
0	0	1	0	1	1
0	0	1	1	0	1
0	0	1	1	1	1
0	1	0	0	0	1
0	1	0	0	1	1
0	1	0	1	0	1
0	1	0	1	1	1
0	1	1	0	0	1
0	1	1	0	1	1
0	1	1	1	0	1
0	1	1	1	1	1
1	0	0	0	0	1
1	0	0	0	1	1
1	0	0	1	0	1
1	0	0	1	1	1
1	0	1	0	0	1
1	0	1	0	1	1
1	0	1	1	0	1
1	0	1	1	1	1
1	1	0	0	0	1
1	1	0	0	1	1
1	1	0	1	0	1
1	1	0	1	1	1
1	1	1	0	0	1
1	1	1	0	1	1
1	1	1	1	0	1
1	1	1	1	1	0

Función
$y=(((BCD)'AE)')'$



(b)

Tabla de Verdad (3b)							
a	b	c	d	e	f	y1	y2
0	0	0	0	0	0	0	0
0	0	0	0	0	1	0	0
0	0	0	0	1	0	1	0
0	0	0	0	1	1	1	1
0	0	0	1	0	0	1	0
0	0	0	1	0	1	1	1
0	0	0	1	1	0	1	0
0	0	0	1	1	1	1	0
0	0	1	0	0	0	1	0
0	0	1	0	0	1	1	1
0	0	1	0	1	0	1	0
0	0	1	0	1	1	1	0
0	0	1	1	0	0	1	0
0	0	1	1	0	1	1	0
0	0	1	1	1	0	1	0
0	0	1	1	1	1	1	0
0	1	0	0	0	0	0	0
0	1	0	0	0	1	0	0
0	1	0	0	1	0	1	0
0	1	0	0	1	1	1	0
0	1	0	1	0	0	1	0
0	1	0	1	0	1	1	0
0	1	0	1	1	0	1	0
0	1	0	1	1	1	1	0
0	1	1	0	0	0	1	0
0	1	1	0	0	1	1	0
0	1	1	0	1	0	1	0
0	1	1	0	1	1	1	0
0	1	1	1	0	0	1	0
0	1	1	1	0	1	1	0
0	1	1	1	1	0	1	0
0	1	1	1	1	1	1	0
1	0	0	0	0	0	1	0
1	0	0	0	0	1	1	0
1	0	0	0	1	0	1	0
1	0	0	0	1	1	1	0
1	0	0	1	0	0	1	0
1	0	0	1	0	1	1	0
1	0	0	1	1	0	1	0
1	0	0	1	1	1	1	0
1	0	1	0	0	0	1	0
1	0	1	0	0	1	1	0
1	0	1	0	1	0	1	0
1	0	1	0	1	1	1	0
1	0	1	1	0	0	1	0
1	0	1	1	0	1	1	0
1	0	1	1	1	0	1	0
1	0	1	1	1	1	1	0
1	1	0	0	0	0	1	0
1	1	0	0	0	1	1	0
1	1	0	0	1	0	1	0
1	1	0	0	1	1	1	0
1	1	0	1	0	0	0	0
1	1	0	1	0	1	0	1
1	1	0	1	1	0	0	0
1	1	0	1	1	1	0	1
1	1	1	0	0	0	0	0
1	1	1	0	0	1	0	1
1	1	1	0	1	0	0	0
1	1	1	0	1	1	0	1
1	1	1	1	0	0	0	0
1	1	1	1	0	1	0	1
1	1	1	1	1	0	0	0
1	1	1	1	1	1	0	1

Función
$y2 = A \oplus (C+D+E)$

4) **Deducción de tabla y función desde diagrama de tiempo: Se interpretaron las señales A, B y C junto con Fs, identificando interferencias indeseadas y deduciendo la función booleana complementada.**

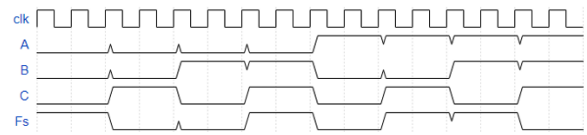


Tabla de Verdad (4)			
X2	X1	X0	S
0	0	0	1
0	0	1	1
0	1	0	1
0	1	1	0
1	0	0	1
1	0	1	0
1	1	0	0
1	1	1	1

Para buscar las funciones aplicamos el mapa de Karnaugh.

A/BC	00	01	11	10
0	1	0	1	0
1	0	1	0	1

Como resultados obtenemos las siguientes funciones para SoP y PoS:

Min-terminos (SoP)
A'B'C' + A'BC + AB'C + ABC'

Max-terminos (PoS)
(A + B + C')(A + B' + C)(A' + B + C)(A' + B' + C')

5) **Multiplicador binario de 2 bits: Se determinaron las líneas de salida necesarias y se obtuvieron las expresiones booleanas simplificadas para cada una, verificando las combinaciones posibles.**

El numero de salidas necesarias para la realización de este circuito es de 4 salidas. Para obtener las funciones necesitamos aplicar la tabla de verdad y el mapa de Karnaugh, como resultados de estas ultimas obtuvimos:

Tabla de Verdad (5)							
A1	A0	B1	B0	S1	S2	S3	S4
0	0	0	0	0	0	0	0
0	0	0	1	0	0	0	0
0	0	1	0	0	0	0	0
0	0	1	1	0	0	0	0
0	1	0	0	0	0	0	0
0	1	0	1	0	0	0	1
0	1	1	0	0	0	1	0
0	1	1	1	0	0	1	1
1	0	0	0	0	0	0	0
1	0	0	1	0	0	1	0
1	0	1	0	0	1	0	0
1	0	1	1	0	1	1	0
1	1	0	0	0	0	0	0
1	1	0	1	0	0	1	1
1	1	1	0	0	1	1	0
1	1	1	1	1	0	0	1

Mapa de Karnaugh: Aquí aplicamos lo de el conjunto de unos para obtener la función más simplificada posible.

AB/CD	00	01	11	10
00	1	1	0	1
01	1	0	1	0
11	0	1	1	1
10	1	0	1	0

Las funciones obtenidas son:

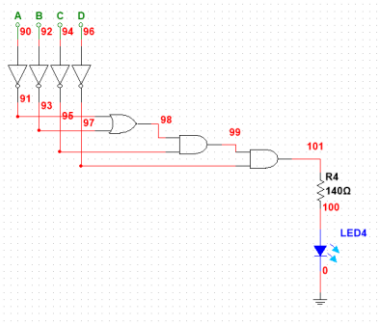
Min-terminos (SoP)
A'B'C' + A'B'D' + A'C'D' + B'C'D' + BCD + ACD + ABD + ABC

Max-terminos (PoS)
(A + B + C' + D')(A + B' + C + D')(A + B' + C' + D)(A' + B + C + D')(A' + B + C' + D)(A' + B' + C + D)

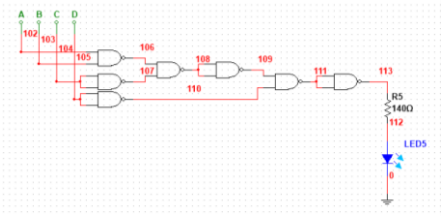
6) **Simulación de una expresión lógica: Se implementó la función original, su equivalente con compuertas NAND y con compuertas NOR, comparando los resultados en Multisim.**

$$Z = \overline{((A + B).C).D}$$

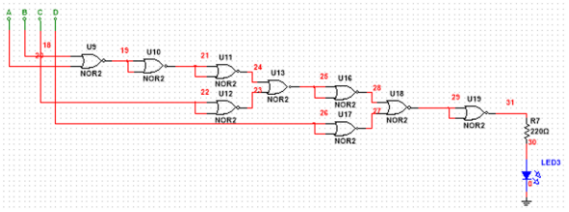
Tal y como esta:



Usando compuertas dual line Nands:



Usando compuertas dual line Nors:



7) **Complementador a dos de 4 bits con indicador de signo: Se obtuvo la tabla de verdad y se derivó la función de salida en forma PoS, además del código en Verilog estructural.**

La tabla de verdad para este circuito es la siguiente:

Tabla de Verdad (7)								
A1	A0	B1	B0	S	S1	S2	S3	S4
0	0	0	0	1	0	0	0	0
0	0	0	1	0	1	1	1	1
0	0	1	0	0	1	1	1	0
0	0	1	1	0	1	1	0	1
0	1	0	0	0	1	1	0	0
0	1	0	1	0	1	0	1	1
0	1	1	0	0	1	0	1	0
0	1	1	1	0	1	0	0	1
1	0	0	0	0	1	0	0	0
1	0	0	1	1	0	1	1	1
1	0	1	0	1	0	1	1	0
1	0	1	1	1	0	1	0	1
1	1	0	0	1	0	1	0	0
1	1	0	1	1	0	0	1	1
1	1	1	0	1	0	0	1	0
1	1	1	1	1	0	0	0	1

Para encontrar las funciones de cada una de las salidas en PoS necesitamos aplicar mapas de Karnaugh, los cuales son:

Salida (S)				
AB/CD	00	01	11	10
00	1	0	0	0
01	0	0	0	0
11	1	1	1	1
10	0	1	1	1

Salida (S1)				
AB/CD	00	01	11	10
00	0	1	1	1
01	1	1	1	1
11	0	0	0	0
10	1	0	0	0

Salida (S2)				
AB/CD	00	01	11	10
00	0	1	1	1
01	1	0	0	0
11	1	0	0	0
10	0	1	1	1

Salida (S3)				
AB/CD	00	01	11	10
00	0	1	0	1
01	0	1	0	1
11	0	1	0	1
10	0	1	0	1

Salida (S4)				
AB/CD	00	01	11	10
00	0	1	1	0
01	0	1	1	0
11	0	1	1	0
10	0	1	1	0

Ya teniendo los mapas podemos calcular las funciones u obtener las funciones simplificadas correspondiente a cada salida:

Para la salida S:

Max-terminos (PoS)
AD + AC + AB + A'B'C'D'

Para la salida S1:

Max-terminos (PoS)
A'D + A'C + A'B + AB'C'D'

Para la salida S2:

Max-terminos (PoS)
B'D + B'C + BC'D'

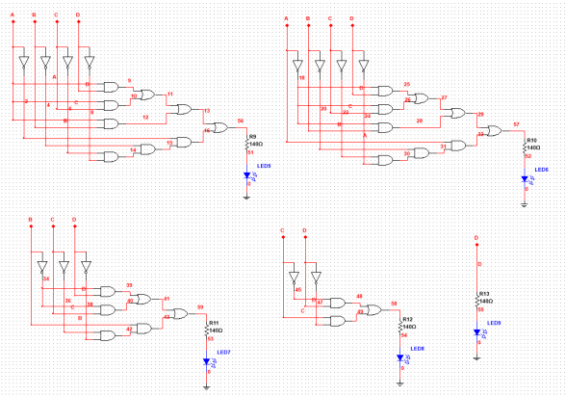
Para la salida S3:

Max-terminos (PoS)
C'D + CD'

Para la salida S4:

Max-terminos (PoS)
D

Luego de obtener las funciones las simule en Multisim porque se me hace más fácil poder escribir el código observando el diagrama del circuito, el cual es:



Luego de programar en HDL (Verilog), validamos que todo funciona correctamente, este es el archivo principal o donde se encuentran las intanciaciones y es el cuerpo del código:

```
module EXERCISE7(
    // Inputs and outputs declaration
    input A0,A1,B0,B1,
    output S, S1,S2, S3 ,S4
);

    //Wires Declaration
    wire nwA, nwB, nwC, nwD; //not wires

    wire ow1, ow2, ow3, ow4, ow5, ow6, ow7; //OR wires;

    wire aw1, aw2, aw3, aw4, aw5, aw6, aw7; //AND wires

    //NOT module declaration
    NOT1 IC_NOT(
         .NE1(A0),              // not 1
         .NE2(A1),              // not 2
         .NE3(B0),              // not 3
         .NE4(B1),              // not 4

         .NS1(nwA),
         .NS2(nwB),
         .NS3(nwC),
         .NS4(nwD)
    );

    // First Circuit
    //AND IC
    AND1 IC_AND1(
        .AE1(A0),.AE2(B1),   // and 1
        .AE3(A0),.AE4(B0),   // and 2
        .AE5(A0),.AE6(A1),   // and 3
        .AE7(nwC),.AE8(nwd),   // and 4

        .AS1(aw1),
        .AS2(aw2),
        .AS3(aw3),
        .AS4(aw4)
    );

        AND2 IC_AND2(
        .AE1(nwB),.AE2(aw4),   // and 5
        .AE3(nwA),.AE4(aw5),   // and 6

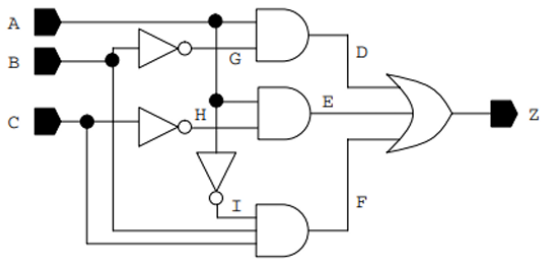
        .AS1(aw5),
        .AS2(aw6),
    );

    //OR module declaration
    // IC1
    OR1 IC_OR1(
        .OE1(aw1),.OE2(aw2),   // or 1
        .OE3(aw7),.OE4(aw3),   // or 2
        .OE5(aw8),.OE6(aw6),   // or 3

        .OS1(aw7),
        .OS2(aw8),
        .OS3(S)
    );

    // Second Circuit
    //AND IC
    AND1 IC_AND1(
        .AE1(A0),.AE2(B1),   // and 1
        .AE3(A0),.AE4(B0),   // and 2
        .AE5(A0),.AE6(A1),   // and 3
        .AE7(nwC),.AE8(nwd),   // and 4
```

8) Complementador a dos de 4 bits con indicador de signo: Se obtuvo la tabla de verdad y se derivó la función de salida en forma PoS, además del código en Verilog estructural.



Se obtuvo la tabla de verdad:

Tabla de Verdad (8)			
A	B	C	Z
0	0	0	1
0	0	1	1
0	1	0	1
0	1	1	0
1	0	0	1
1	0	1	0
1	1	0	0
1	1	1	1

Luego el mapa de Karnaugh para la función:

A/BC	00	01	11	10
0	1	0	1	0
1	0	1	0	1

Codigo verilog del circuito:

```
module EXERCISE8(
    input A, B, C,
    output X
);

    //Wires Declaration
    wire nwA, nwB, nwC, nwD; //not wires
    wire w1, w2, w3, w4;

    //NOT module declaration
    NOT1 IC_NOT(
         .NE1(A),               // not 1
         .NE2(B),               // not 2
         .NE3(C),               // not 3

         .NS1(nwA),
         .NS2(nwB),
         .NS3(nwC)
    );

        AND1 IC_AND1(
        .AE1(nwB),.AE2(A),            // and 1
        .AE3(A),.AE4(nwC),            // and 2
        .AE5(nwA),.AE6(B),.AE7(C),    // and 3

        .AS1(w1),
        .AS2(w2),
        .AS3(w3)
    );

    OR1 IC_OR1(
        .OE1(w1),.OE2(w2),.OE3(w3),   // or 1

        .OS1(X)
    );
endmodule
```

Testbench:

```
module EXERCISE8_TESTBENCH(
    reg A, B, C,
    wire X;

    EXERCISE8 uut(
        .A(A),
        .B(B),
        .C(C),
        .D(D),
        .X(X)
    );

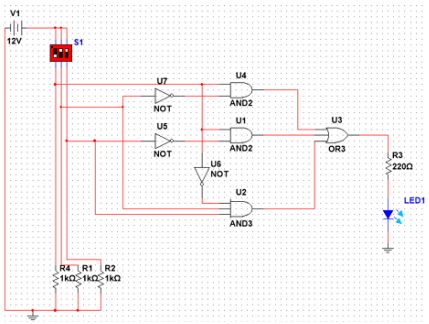
    initial begin
        //ENCABEZADO
        $display("A B C | %b", A, B, C, X);

        //Pruebas
        A = 0; B = 0; C = 0; #10;
        A = 0; B = 0; C = 1; #10;
        A = 0; B = 1; C = 0; #10;
        A = 0; B = 1; C = 1; #10;
        A = 1; B = 0; C = 0; #10;
        A = 1; B = 0; C = 1; #10;
        A = 1; B = 1; C = 0; #10;
        A = 1; B = 1; C = 1; #10;

        $finish;
);
endmodule
```

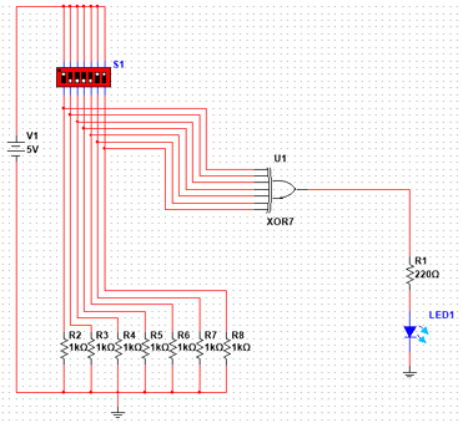
```
A B C | Z
0 0 0 | 0
0 0 1 | 0
0 1 0 | 0
0 1 1 | 1
1 0 0 | 1
1 0 1 | 1
1 1 0 | 1
1 1 1 | 0
testbench.sv:33: $finish called at 80 (1s)
Done!
```

Diagrama del circuito en Multisim:



9) Detector de paridad de 7 bits: Se diseñó y simuló el circuito lógico en Multisim y se implementó en Verilog, comprobando su comportamiento con diferentes combinaciones.

Diagrama en multisim:



Codigo verilog:

```
module EXERCISE9(
    input [0:5] A, B,
    output X
);

    //xor module declaration
    xor XOR1 (
          .NE1(B),              // not 1
          .NE2(A[0]),           // not 2
          .NE3(A[1]),
          .NE4(A[2]),
          .NE5(A[3]),
          .NE6(A[4]),
          .NE7(A[5]),

          .NS1(X),
          .NS2(nwB),
          .NS3(nwC)
    );

endmodule
```

10) Control de semáforos en rotonda: Se diseñó el sistema con prioridad jerárquica de sensores, se obtuvo la tabla de verdad, se simplificó por K-Maps, se simuló el diagrama lógico y se escribió el código HDL para implementación en FPGA.

Tabla de verdad:

Tabla de Verdad (10)							
P0	P1	P2	P3	S0	S1	S2	S3
0	0	0	0	0	0	0	0
0	0	0	1	1	1	0	1
0	0	1	0	1	0	1	1
0	0	1	1	1	1	1	0
0	1	0	0	0	1	1	1
0	1	0	1	1	1	1	0
0	1	1	0	1	1	0	1
0	1	1	1	1	1	0	0
1	0	0	0	1	1	1	0
1	0	0	1	1	1	1	0
1	0	1	0	1	1	0	1
1	0	1	1	1	1	0	0
1	1	0	0	1	0	1	1
1	1	0	1	1	0	1	0
1	1	1	0	1	0	0	1
1	1	1	1	1	0	0	0

K-maps:

Salida (S0)				
AB/CD	00	01	11	10
00	0	1	1	1
01	0	1	1	1
11	1	1	1	1
10	1	1	1	1

Salida (S1)				
AB/CD	00	01	11	10
00	0	1	1	0
01	1	1	1	1
11	0	0	0	0
10	1	1	1	1

Salida (S2)				
AB/CD	00	01	11	10
00	0	0	1	1
01	1	1	0	0
11	1	1	0	0
10	1	1	0	0

Salida (S3)				
AB/CD	00	01	11	10
00	0	1	0	1
01	1	0	0	1
11	1	0	0	1
10	0	0	0	1

Funciones simplificadas (están respectivamente ordenadas con los k-maps anteriores):

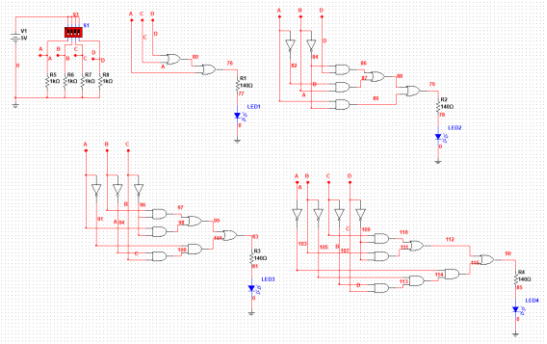
Min-terminos (SoP)
D + C + A

Min-terminos (SoP)
B'D + A'B + AB'

Min-terminos (SoP)
BC' + AC' + A'B'C

Min-terminos (SoP)
CD' + BD' + A'B'C'D

Diagrama lógico:



IV. MONTAJE EN FPGA

El circuito número 10 fue diseñado y simulado para implementarse en la FPGA con GoWin mediante la descripción estructural en Verilog. Se verificaron los resultados mediante unas las salidas físicas de los pines de la tarjeta. Los LEDs de salida mostraron las condiciones lógicas esperadas, demostrando que las funciones teóricas y simuladas coinciden con el comportamiento real del hardware. Esta etapa permitió observar la transición del diseño lógico desde un nivel de simulación hasta su ejecución física, confirmando la correcta asignación de pines y la integridad del diseño combinacional.

Para realizar el montaje en físico debemos implementar el código HDL, el cual es el siguiente:

```
module TRAFFIC_LIGHT(
    // Inputs and outputs declaration
    input A,B,C,D,
    output S0,S1, S2 ,S3
);

    //Wires Declaration
    wire nwA, nwB, nwC, nwD; //not wires

    wire ow1, ow2, ow3, ow4;  //OR wires

    wire aw1, aw2, aw3, aw4, aw5, aw6, aw7, //AND wires
         aw8, aw9, aw10, aw11, aw12;

    //NOT module declaration
    NOT1 IC_NOT(
         .NE1(A),               // not 1
         .NE2(B),               // not 2
         .NE3(C),               // not 3
         .NE4(D),               // not 4

         .NS1(nwA),
         .NS2(nwB),
         .NS3(nwC),
         .NS4(nwD)
    );

    //OR module declaration
    // IC1
    OR1 IC_OR1(
        .OE1(C),.OE2(D),       // or 1
        .OE3(ow1),.OE4(A),     // or 2
        .OE5(aw1),.OE6(aw2),   // or 3
        .OE7(ow2),.OE8(aw3),   // or 4

        .OS1(ow1),
        .OS2(S0),
        .OS3(ow2),
        .OS4(S1)
    );

    // IC2
    OR2 IC_OR2(
        .OE1(aw4),.OE2(aw5),   // or 5
        .OE3(ow3),.OE4(aw7),   // or 6
        .OE5(aw8),.OE6(aw9),   // or 7
        .OE7(ow4),.OE8(aw12),       // or 8

        .OS1(ow3),
        .OS2(S2),
        .OS3(ow4),
        .OS4(S3)
    );
```

```
    //AND module declaration
    // IC1
    AND1 IC_AND1(
        .AE1(nwB),.AE2(D),    // and 1
        .AE3(nwA),.AE4(B),    // and 2
        .AE5(A),.AE6(nwB),    // and 3
        .AE7(B),.AE8(nwC),    // and 4

        .AS1(aw1),
        .AS2(aw2),
        .AS3(aw3),
        .AS4(aw4)
    );

    // IC2
    AND2 IC_AND2(
        .AE1(A),.AE2(nwC),    // and 5
        .AE3(nwB),.AE4(C),    // and 6
        .AE5(aw6),.AE6(nwA),   // and 7
        .AE7(C),.AE8(nwD),    // and 8

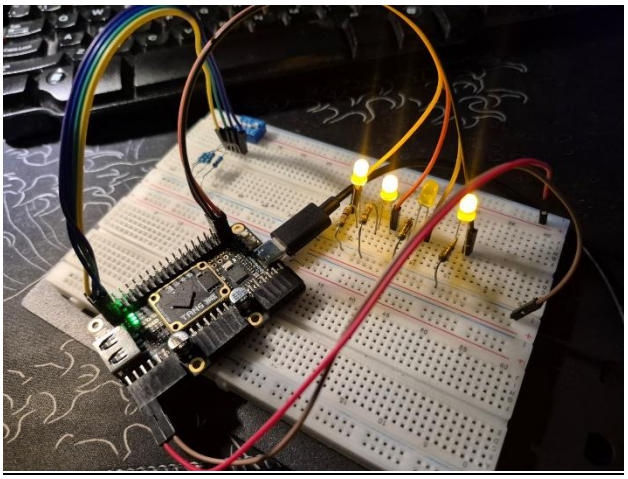
        .AS1(aw5),
        .AS2(aw6),
        .AS3(aw7),
        .AS4(aw8)
    );

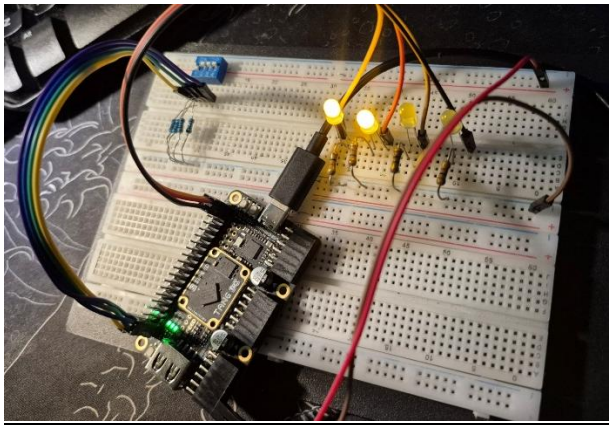
    // IC3
    AND3 IC_AND3(
        .AE1(B),.AE2(nwD),    // and 9
        .AE3(nwC),.AE4(D),    // and 10
        .AE5(nwB),.AE6(aw10),    // and 11
        .AE7(aw11),.AE8(nwA),    // and 12

        .AS1(aw9),
        .AS2(aw10),
        .AS3(aw11),
        .AS4(aw12)
    );

endmodule
```

FPGA con el código implementado y en funcionamiento (Prueba 1 y 2):





Enlace al video sobre practica 10:

https://itlaedudo-my.sharepoint.com/:v:/g/personal/20240684_itla_edu_do/EQLrZdCUzCJFs16teBgSc_MBm6a2o1sJuTLbDjipKXhvQg?e=lJn4TF

V. Conclusion

La práctica permitió consolidar conocimientos fundamentales del diseño digital, desde la formulación de tablas de verdad hasta la implementación física en FPGA.
Se comprobó que el uso de compuertas lógicas, mapas de Karnaugh, formas canónicas (SoP/PoS) y Verilog permite construir sistemas eficientes y verificables tanto en simulación como en hardware real.
El uso de Multisim facilitó la validación de los circuitos antes de su implementación, mientras que GoWin IDE proporcionó una plataforma adecuada para probar los diseños en FPGA.
En general, la experiencia reforzó la comprensión del diseño combinacional, el análisis lógico y la importancia de las herramientas digitales modernas para el desarrollo de sistemas electrónicos. En general, esta práctica reforzó la importancia de los métodos de simplificación booleana, la utilidad del diseño estructural en Verilog y la aplicación de compuertas digitales en sistemas combinacionales de control.

VI. Referencias

https://en.wikipedia.org/wiki/Digital_electronics
https://naylampmechatronics.com/blog/39_resistencias-pull-up-y-pull-down.html
https://es.wikipedia.org/wiki/Sistema_de_numeraci%C3%B3n
https://es.wikipedia.org/wiki/C%C3%B3digo_binario
https://es.slideshare.net/slideshow/sistemas-de-codigos-en-la-electronica-digital/158426000